

Building and Deploying a Python MCP Server for DR Plan Generation

This tutorial shows how to create a **Model Context Protocol (MCP) server** in Python that calls a disaster recovery (DR) plan endpoint, caches the result, and lets a user iteratively edit the plan before saving it. We'll use either the **FastMCP** framework or Cohere's **NorthMCPServer** (which extends FastMCP) to implement the MCP server. The server will be containerized with Docker and deployed on Kubernetes. We'll test it locally and via an MCP client (using tools like **mcp-use** or the FastMCP client). Throughout, we use Python, Redis (as a cache), and MongoDB (for final persistence).

An MCP server “exposes tools, functions, or external data sources to AI models through a standardized protocol” ¹. In practice, it runs an HTTP/SSE (or WebSocket) API that an LLM-based client (like Claude Desktop) calls to invoke **tools** (functions) defined on the server. In our case, the server will have tools to generate a DR plan (by calling an external service) and to view/edit that plan.

1. Setup and Dependencies

1. **Python environment:** Create a Python 3 virtual environment and install required packages. At minimum you'll need:
2. `fastmcp>=0.1.0` (or the original `mcp[cli]`)
3. `starlette` and `uvicorn` (for HTTP serving)
4. `north-mcp-python-sdk` if using North's server (for auth) ².
5. `redis` and `motor` (or `pymongo`) for Redis and MongoDB access.
6. HTTP client libraries like `httpx` or `requests` for calling the DR endpoint.

For example:

```
python3 -m venv venv
source venv/bin/activate
pip install fastmcp starlette uvicorn httpx redis motor north-mcp-python-sdk
```

1. **Running Redis and MongoDB locally:** You can run the latest Redis and MongoDB as Docker containers or via your system's package manager. For example, using Docker:

```
docker run -d --name redis -p 6379:6379 redis:latest
docker run -d --name mongo -p 27017:27017 mongo:latest
```

Redis will serve as a **cache/session store**, and MongoDB will hold the final plans. (Using a cache is recommended when the server “maintains state or needs persistence” ³.)

2. Writing the MCP Server

2.1 Choosing FastMCP or NorthMCP

- **FastMCP:** A lightweight MCP SDK. Tools are defined with `@mcp.tool()` decorators on async Python functions ⁴. For example:

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("DR-Plan-Server")
@mcp.tool()
async def add(a: int, b: int) -> int:
    return a + b
```

This automatically exposes an `add` tool. See the Northflank blog example: they define `mcp = FastMCP("Example MCP")` with `@mcp.tool()` functions ⁴.

- **NorthMCPServer:** Extends FastMCP with built-in auth. You create it similarly, but you pass a shared secret:

```
from north_mcp_python_sdk import NorthMCPServer

mcp = NorthMCPServer(name="DR-Plan-Server", port=5222,
    server_secret="MY_SECRET")
```

This tells the server to only accept requests whose Bearer token includes that `server_secret` ⁵. The North SDK also lets you extract the authenticated user or OAuth tokens from the header ² ⁵.

In this tutorial we'll illustrate with **FastMCP** for simplicity (it works without any special tokens), but note that switching to North's server just adds authentication. The concepts below are the same. For example, the North MCP SDK supports the Streamable HTTP transport (instead of SSE) and enforces a shared secret in the Bearer header ². When enabled, it will parse a header like `Authorization: Bearer <base64-json>` and extract fields like `server_secret` and `user_id_token` ⁶.

2.2 Defining the Application Code

Here's a high-level sketch of the server (`app.py`). We use Starlette as the HTTP framework, but FastMCP can run with any ASGI app.

```
from starlette.applications import Starlette
from starlette.routing import Route
from starlette.responses import JSONResponse, PlainTextResponse
from starlette.requests import Request
```

```

import httpx, json
import redis
import motor.motor_asyncio

from mcp.server.fastmcp import FastMCP
# If using North: from north_mcp_python_sdk import NorthMCPServer

# Initialize FastMCP or NorthMCPServer
mcp = FastMCP(name="DR-Plan-Server")
# For North:
# mcp = NorthMCPServer(name="DR-Plan-Server", port=8000,
server_secret="MY_SECRET")

# Connect to Redis and MongoDB
redis_client = redis.Redis(host='localhost', port=6379, db=0)
mongo_client = motor.motor_asyncio.AsyncIOMotorClient("mongodb://localhost:
27017")
mongo_db = mongo_client["drplans"]
mongo_collection = mongo_db["plans"]

# Tool: Call external DR plan generation (mocked for now)
@mcp.tool()
async def generate_plan(pattern_ids: list[str]) -> dict:
    """
    Call the DR plan generation API with the given pattern IDs and return the
    plan (as a dict).
    """
    # Mocked call to external service:
    async with httpx.AsyncClient() as client:
        response = await client.post("http://mocked-dr-service/plan",
json={"pattern_id": pattern_ids})
        plan = response.json() # Expected to follow the given schema.

# Cache the plan in Redis, keyed by user or session. Here we assume a single
session:
    redis_client.set("current_plan", json.dumps(plan))
    return plan

# Tool: View current plan (from cache)
@mcp.tool()
async def get_current_plan() -> dict:
    """
    Retrieve the current DR plan from Redis and return it.
    """
    data = redis_client.get("current_plan")
    return json.loads(data) if data else {}

# Tool: Update the plan text (a simple example of editing)

```

```

@mcp.tool()
async def update_plan_section(section: str, content: str) -> dict:
    """
    Update a top-level field in the plan (e.g. name, notes) or add to steps.
    """
    plan = json.loads(redis_client.get("current_plan"))
    if section in plan:
        plan[section] = content
    # Save back to Redis:
    redis_client.set("current_plan", json.dumps(plan))
    return plan

# Tool: Add a recovery step (example)
@mcp.tool()
async def add_recovery_step(task_id: int, task: str, description: str) -> dict:
    """
    Add a new task to recoverySteps->preExercise for illustration.
    """
    plan = json.loads(redis_client.get("current_plan"))
    step = {
        "task_id": task_id,
        "task": task,
        "short_description": description,
        "prerequisites": [],
        "dependencies": [],
        "description": "",
        "expectedResult": "",
        "expectedTimeToComplete": "",
        "in_disaster": True,
        "in_exercise": True,
        "assignment_group": ""
    }
    plan.setdefault("disasterRecoverySteps", {}).setdefault("preExercise",
[]).append(step)
    redis_client.set("current_plan", json.dumps(plan))
    return plan

# Tool: Finalize and save plan to MongoDB
@mcp.tool()
async def save_plan() -> str:
    """
    Persist the current plan from Redis into MongoDB, and return a confirmation.
    """
    plan = json.loads(redis_client.get("current_plan"))
    if plan:
        await mongo_collection.insert_one(plan)
        return "Plan saved to MongoDB."
    else:

```

```

        return "No plan to save."

# HTTP handlers for MCP protocol
async def mcp_handler(request: Request):
    payload = await request.json()
    result = await mcp.handle_http(payload)    # FastMCP's HTTP handler
    return JSONResponse(result)

async def health(request: Request):
    return PlainTextResponse("ok", status_code=200)

app = Starlette(routes=[
    Route("/mcp", endpoint=mcp_handler, methods=["POST"]),
    Route("/health", endpoint=health, methods=["GET"])
])

```

- **Tools explanation:** We defined tools decorated with `@mcp.tool()`. For example, `generate_plan` calls an external DR endpoint (mocked here) and caches the result in Redis. The editing tools (`update_plan_section`, `add_recovery_step`) simply load the JSON plan from Redis, modify it, and save it back. Finally, `save_plan` inserts the current plan into MongoDB. In a real app you would add error handling, authorization checks, and more sophisticated merging logic.

- **Authorization header:** If using **NorthMCPServer**, you would require clients to send a Bearer token containing your `server_secret`. The North SDK will automatically parse and validate the token. (For example, setting `NorthMCPServer(..., server_secret="secret")` makes the server reject requests missing that secret ⁵.) The Bearer token is expected to be a base64-encoded JSON with fields like `"server_secret"` and `"user_id_token"`. In debug logs, NorthMCPServer shows e.g. "Has server_secret: True, Has user_id_token: True..." ⁶. You can then call `get_authenticated_user()` within tools to inspect the user's identity or connector tokens if needed.

2.3 Capturing the DR Plan Schema

The DR endpoint returns JSON like:

```

{
  "pattern-id": ["id1", "id2"],
  "name": "Name for steps",
  "prerequisites": [...],
  "notes": "Additional notes.",
  "disasterRecoverySteps": {
    "preExercise": [
      {
        "task_id": 1,
        "task": "Task name",
        "short_description": "Short desc",

```

```

        "prerequisites": [],
        "dependencies": [],
        "description": "How to execute",
        "expectedResult": "Expected result",
        "expectedTimeToComplete": "1h",
        "in_disaster": true,
        "in_exercise": false,
        "assignment_group": "Team A"
    }
],
"recoverySteps": [],
"serviceValidation": [],
"postExercise": []
}
}

```

We can manipulate this object in Python (as `dict`s and `list`s). For clarity, one can define Pydantic models for strict validation, but it isn't strictly required since MCP will pass JSON data through. The FastMCP framework will automatically convert Python dicts/lists into valid MCP responses.

2.4 Running the Server Locally

To test locally, just run the Python app. For FastMCP, it automatically uses Uvicorn under the hood. For example:

```
uvicorn app:app --host 0.0.0.0 --port 8080
```

This starts an HTTP server on port 8080 with endpoints `/mcp` (MCP API) and `/health`. (Our Dockerfile below uses `CMD ["uvicorn", "app:app", ...]` so the same command will run inside Docker.)

You can verify it's running by hitting the health endpoint:

```
curl http://localhost:8080/health
```

should return `ok`.

3. Dockerizing the MCP Server

Create a `Dockerfile` to containerize the server. For example, using the guidance from Northflank's blog

7 8 :

```

# Dockerfile
FROM python:3.12-slim

```

```

WORKDIR /app
# Install any system dependencies (if needed for your tools)
RUN apt-get update -y && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/*
# Copy and install Python deps
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
# Expose port (FastMCP uses env PORT or default 8080)
ENV PORT=8080
EXPOSE 8080
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080"]

```

Here `requirements.txt` should list `fastmcp`, `starlette`, `uvicorn`, `httpx`, `redis`, `motor`, etc (and `north-mcp-python-sdk` if using North). The Dockerfile uses a slim Python base and installs updates (see ⁷). Then we run Uvicorn to serve our Starlette app on port 8080 (the code in the Container uses the `PORT` env). These steps match the example shown above (e.g. "FROM python:3.12-slim" and `pip install` ⁷).

Build the image:

```
docker build -t mcp-dr-plan-server:latest .
```

And run it (linking to Redis and Mongo if needed):

```
docker run -d --name mcp-server -p 8080:8080 mcp-dr-plan-server:latest
```

Now the MCP server is running in Docker.

4. Deploying to Kubernetes

To deploy on Kubernetes, first push your Docker image to a registry. Then create Kubernetes manifests, for example:

```

# k8s-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mcp-dr-plan
spec:
  replicas: 1
  selector:
    matchLabels: {app: mcp-dr-plan}

```

```

template:
  metadata:
    labels: {app: mcp-dr-plan}
  spec:
    containers:
      - name: mcp-server
        image: myregistry/mcp-dr-plan-server:latest
        ports:
          - containerPort: 8080
        env:
          - name: SERVER_SECRET
            valueFrom:
              secretKeyRef: {name: mcp-secret, key: server_secret}
          # (Add other secrets or config as needed for Redis/Mongo credentials)

```

```

# k8s-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: mcp-service
spec:
  selector:
    app: mcp-dr-plan
  ports:
    - name: http
      port: 80
      targetPort: 8080

```

Apply these with `kubectl apply -f k8s-deployment.yaml -f k8s-service.yaml`. This sets up a Deployment with your container and a Service exposing it. If you use namespace secrets (for example `mcp-secret` holding the `server_secret`), Kubernetes will inject them. You would also typically deploy Redis and MongoDB (or connect to managed instances) and inject their connection strings as environment variables or secrets.

Optionally, add a Kubernetes `Ingress` or use your cloud's load balancer to expose the service publicly. The Northflank blog suggests an HTTPS ingress or health checks, but the core idea is the same: your Kubernetes setup should route `/mcp` to the container on port 8080.

5. Testing the MCP Server

Once running, you can test the MCP server with an MCP-compatible client. For example, using FastMCP's Python client (as in PondHouse's guide) ⁹ ¹⁰ :


```

from fastmcp.client import Client
import asyncio

async def test():
    client = Client("http://localhost:8080/sse") # SSE transport endpoint
    async with client:
        print("Connected:", client.is_connected())
        tools = await client.list_tools()
        print("Tools:", [t.name for t in tools])
        # Call the generate_plan tool
        result = await client.call_tool("generate_plan", {"pattern_ids":
["abc123"]})
        print("Generated Plan:", result)

```

This code connects to our `/mcp` endpoint (note the `/sse` suffix for Streamable HTTP) and lists available tools ⁹. It then invokes the `generate_plan` tool we defined, passing some example `pattern_ids`. You should see the returned JSON plan. Similarly, you could call `update_plan_section`, `add_recovery_step`, etc. The cited example shows listing tools and calling a tool named `"edit_file"` ⁹ ¹⁰; in our case you'd call your tools by name.

Alternatively, you can use **mcp-use** to simulate a multi-turn chat. The `mcp-use` library (or its CLI) lets you point an agent (like GPT-4o via LangChain) at your MCP server. For example, in Python:

```

from mcp_use import MCPClient, MCPAgent
from langchain_openai import ChatOpenAI
import asyncio, os
from dotenv import load_dotenv

async def run_agent():
    load_dotenv()
    config = {"mcpServers": {
        "dr_server": {"command": "", "protocol": "sse", "url": "http://
localhost:8080/mcp"}
    }}
    client = MCPClient.from_dict(config)
    llm = ChatOpenAI(model="gpt-4o")
    agent = MCPAgent(llm=llm, client=client, max_steps=10)
    async for chunk in agent.astream("Generate a disaster recovery plan for
system X, then refine it by adding an extra task"):
        print(chunk.text, end="")

```

This would let an LLM chain query your MCP server tools in multiple turns (listening for tool calls and passing results back to the conversation). `mcp-use` abstracts the interaction so the LLM can say “call tool `generate_plan` with `X`” and then process the JSON result. (See [47] for how the FastMCP client works – `mcp-use` works similarly under the hood.)

Lastly, you can use tools like the **MCP Inspector** (a web-based debug tool) to manually craft MCP messages for testing. But the above Python clients demonstrate how to connect programmatically.

6. Caching and Persistence

In the above code, we used **Redis** to cache the current DR plan between edits. This is important because each MCP call is stateless; we need a place to keep the plan as the user refines it. Redis is well-suited as an in-memory cache ³. We store the plan JSON under a key like `"current_plan"`, or better, key by user ID from the auth token. The server's tools then fetch/modify this cached plan on each call.

When the user is done refining the plan (perhaps they call a finalizing tool like `save_plan`), we insert it into **MongoDB** for permanent storage. MongoDB is a natural choice for JSON-like documents. In code we used Motor (async PyMongo) to call `insert_one(plan)`. The official MongoDB blog notes that MCP servers can directly interface with MongoDB to explore or update data, so storing JSON documents is well-aligned with the protocol (and MongoDB even has an official MCP server) ¹¹.

To recap the flow: - **Initial generation:** Client calls `generate_plan`, server calls the DR API and caches the result. Plan JSON is returned to client. - **Editing rounds:** Client calls `get_current_plan` or editing tools. Each call fetches JSON from Redis, updates it (in Python) based on tool parameters (e.g. adding a step), then saves back to Redis and returns the updated JSON. - **Finalization:** Client calls `save_plan` (or similar), and the server writes the cached JSON into MongoDB.

7. Summary

We have built a Python MCP server with FastMCP (or NorthMCPServer) that: - Defines tools for generating and editing a disaster recovery plan. - Uses Redis to cache the plan during a multi-turn conversation. - Uses MongoDB for final persistence. - Runs in Docker with a straightforward `Dockerfile` ⁷ ⁸. - Can be deployed on Kubernetes via a Deployment and Service. - Can be tested locally via an MCP client (as shown above ⁹ ¹⁰) or with mcp-use agents.

This setup gives you a reusable template: replace the DR endpoints and schema with your actual logic, adjust the tools for your needs, and ensure proper authentication (with NorthMCP or other middleware) for security. MCP's standardization means any MCP-compatible client or LLM-based agent can then use your server's tools safely and easily.

Sources: We drew on FastMCP examples and guides ⁴ ⁹ as well as Cohere's NorthMCP SDK documentation ² ⁵. The Northflank blog provides a sample MCP app and Dockerfile structure ⁷ ⁸. The PondHouse guide shows how to connect a FastMCP client and call tools ⁹ ¹⁰. In all cases, best practices like using a cache (Redis) for state are recommended ³.

¹ ³ ⁴ ⁷ ⁸ How to build and deploy a Model Context Protocol (MCP) server | Blog — Northflank
<https://northflank.com/blog/how-to-build-and-deploy-a-model-context-protocol-mcp-server>

² ⁵ ⁶ GitHub - cohere-ai/north-mcp-python-sdk: An sdk for creating MCP Servers with north
<https://github.com/cohere-ai/north-mcp-python-sdk>

9 10 Creating an MCP Server Using FastMCP: A Comprehensive Guide

<https://www.pondhouse-data.com/blog/create-mcp-server-with-fastmcp>

11 Announcing the MongoDB MCP Server | MongoDB Blog

<https://www.mongodb.com/company/blog/announcing-mongodb-mcp-server>